```
import random
import collections
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
print("All libraries imported successfully ✅")
```

```
All libraries imported successfully ✅
```

```
# Task 1

a = 17
b = 5

print(f"a + b = {a + b}")
print(f"a - b = {a - b}")
print(f"a * b = {a * b}")
print(f"a / b = {a / b}")
print(f"a // b = {a // b}")
print(f"a % b = {a % b}")
print(f"a ** b = {a ** b}")


print(type(a))
print(type(a/b))

# Comment: why is the type of a/b different from the type of a?
# Answer: Beacuse of division operator, even if the number are int but division operator always give answer in float.
```

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
a // b = 3
a % b = 2
a ** b = 1419857
<class 'int'>
<class 'float'>
```

```
# Task 2
```

```
p = True
q = False

print(p and q)
print(p or q)
print(not p)
print(p != q)

score = 73

if score >= 85:
  print("Distinction")
elif 70 <= score <= 84:
  print("Merit")
elif 50 <= score <= 69:
  print("Pass")
else:
  print("Fail")
```

```
False
True
False
True
Merit
```

```
# Task 3

sentence = "  Python is a powerful programming language  "

print(sentence.strip())
print(sentence.strip().upper())
print(sentence.strip().count('a'))
print(sentence.strip().replace("powerful", "versatile"))
print(sentence.split())
```

```
Python is a powerful programming language
PYTHON IS A POWERFUL PROGRAMMING LANGUAGE
4
Python is a versatile programming language
['Python', 'is', 'a', 'powerful', 'programming', 'language']
```

```
# Task 4

a = []

for c in range(1, 31):
  if c % 2 == 0:
    a.append(c)
print(*a)

total = 0
number = 1

while number <= 99:
  total += number
  number += 2


print(total)


for x in range(1, 22):
  if x % 3 == 0 and x % 5 == 0:
    print("FizzBuzz")
  elif x % 3 == 0:
    print("Fizz")
  elif x % 5 == 0:
    print("Buzz")
```

```
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
2500
Fizz
Buzz
Fizz
Fizz
Buzz
Fizz
FizzBuzz
Fizz
Buzz
Fizz
```

```
# Task 5

fruits = ["mango", "banana", "apple", "cherry", "orange"]
```

```
print(fruits[0])
print(fruits[-1])

fruits.append("grapes")
print(fruits)

fruits.insert(2, "kiwi")
print(fruits)

fruits.remove("banana")
print(fruits)

fruits.pop(0)
print(fruits)

fruits.sort()
print(fruits)

fruits.reverse()
print(fruits)
```

```
mango
orange
['mango', 'banana', 'apple', 'cherry', 'orange', 'grapes']
['mango', 'banana', 'kiwi', 'apple', 'cherry', 'orange', 'grapes']
['mango', 'kiwi', 'apple', 'cherry', 'orange', 'grapes']
['kiwi', 'apple', 'cherry', 'orange', 'grapes']
['apple', 'cherry', 'grapes', 'kiwi', 'orange']
['orange', 'kiwi', 'grapes', 'cherry', 'apple']
```

```
# Task 6

numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

print(numbers[0:4])

print(numbers[-3:])

print(numbers[2:8])

print(numbers[::2])
```

```
print(numbers[::-1])

print(numbers[:-6:-1])
```

```
[0, 1, 2, 3]
[7, 8, 9]
[2, 3, 4, 5, 6, 7]
[0, 2, 4, 6, 8]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
[9, 8, 7, 6, 5]
```

```
# Task 7

a = [x**2 for x in range(0, 16)]
print(a)

b = [y for y in range(1, 51) if (y % 3 == 0) != (y % 5 == 0)]
print(b)

sentence = "the quick brown fox jumps over the lazy dog"
words = [len(w) for w in sentence.split()]
print(words)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225]
[3, 5, 6, 9, 10, 12, 18, 20, 21, 24, 25, 27, 33, 35, 36, 39, 40, 42, 48, 50]
[3, 5, 5, 3, 5, 4, 3, 4, 3]
```

```
# Task 8

import collections
import random

matrix = [[1, 2, 3],
          [4, 5, 6],
          [7, 8, 9]]

print(matrix[1])
print(matrix[2][1])

transposed = [[matrix[r][c] for r in range(3)] for c in range(3)]
```

```
print(transposed)

random.seed(42)

random_numbers = [random.randint(1, 10) for _ in range(300)]
counts = collections.Counter(random_numbers)

print(counts.most_common(3))
print(counts.most_common()[-1])
```

```
[4, 5, 6]
8
[[1, 4, 7], [2, 5, 8], [3, 6, 9]]
[(4, 42), (2, 40), (9, 35)]
(6, 22)
```

```
# Task 9

student = ("Alice", 22, "Computer Science", 3.8)

print(student[0], student[3])

try:
    student[0] = "Bob"
except TypeError:
    print("Tuples are immutable!")

name, age, major, gpa = student
print(name)
print(age)
print(major)
print(gpa)


cities = [("Karachi", 24.86, 67.01),
          ("Lahore", 31.55, 74.35),
          ("Islamabad", 33.72, 73.06),
          ("Peshawar", 34.01, 71.57),
          ("Quetta", 30.18, 66.99)]

for city, lat, lon in cities:
    print(f"City: {city} | Lat: {lat} | Lon: {lon}")
```

```
sorted_cities = sorted(cities, key=lambda c: c[1])
print(sorted_cities)
```

```
Alice 3.8
Tuples are immutable!
Alice
22
Computer Science
3.8
City: Karachi | Lat: 24.86 | Lon: 67.01
City: Lahore | Lat: 31.55 | Lon: 74.35
City: Islamabad | Lat: 33.72 | Lon: 73.06
City: Peshawar | Lat: 34.01 | Lon: 71.57
City: Quetta | Lat: 30.18 | Lon: 66.99
[('Karachi', 24.86, 67.01), ('Quetta', 30.18, 66.99), ('Lahore', 31.55, 74.35), ('Islamabad', 33.72, 73.06), ('Peshawar', 34.01, 71.57)]
```

```
# Task 10

student = {
    "name": "Sara",
    "age": 20,
    "courses": ["Math", "Physics", "CS"],
    "grades": {"Math": 88, "Physics": 75, "CS": 92}
}

print(student["name"], student["age"])

print(student["courses"])

print(student["grades"]["CS"])

student["year"] = 2

student["age"] = 21

student["grades"].pop("Physics")

for key, value in student.items():
    print(f"{key}: {value}")

print(list(student.keys()))
print(list(student.values()))
```

```
Sara 20
['Math', 'Physics', 'CS']
92
name: Sara
age: 21
courses: ['Math', 'Physics', 'CS']
grades: {'Math': 88, 'CS': 92}
year: 2
['name', 'age', 'courses', 'grades', 'year']
['Sara', 21, ['Math', 'Physics', 'CS'], {'Math': 88, 'CS': 92}, 2]
```

```
# Task 11

cubes = {x: x**3 for x in range(1, 11)}
print(cubes)

even_odd = {x: "even" if x % 2 == 0 else "odd" for x in range(1, 21)}
print(even_odd)

text = "to be or not to be that is the question to be is to think"

freq = {}
for word in text.lower().split():
    freq[word] = freq.get(word, 0) + 1

sorted_freq = sorted(freq.items(), key=lambda x: x[1], reverse=True)
print(sorted_freq)
```

```
{1: 1, 2: 8, 3: 27, 4: 64, 5: 125, 6: 216, 7: 343, 8: 512, 9: 729, 10: 1000}
{1: 'odd', 2: 'even', 3: 'odd', 4: 'even', 5: 'odd', 6: 'even', 7: 'odd', 8: 'even', 9: 'odd', 10: 'even', 11: 'odd', 12: 'even', 13: 'odd', 14: 'even', 15: 'odd', 16:
[('to', 4), ('be', 3), ('is', 2), ('or', 1), ('not', 1), ('that', 1), ('the', 1), ('question', 1), ('think', 1)]
```

```
# Task 12

python_students = {"Alice", "Bob", "Carol", "David", "Eve"}
java_students = {"Bob", "David", "Frank", "Grace", "Alice"}

print(python_students & java_students)

print(python_students | java_students)

print(python_students - java_students)
```

```
print(python_students ^ java_students)

print({"Bob", "Alice"}.issubset(python_students))

python_students.add("Henry")
python_students.remove("Carol")
print(python_students)

nums = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
print(list(set(nums)))
```

```
{'Bob', 'David', 'Alice'}
{'Bob', 'Carol', 'Frank', 'Eve', 'Alice', 'David', 'Grace'}
{'Eve', 'Carol'}
{'Frank', 'Eve', 'Carol', 'Grace'}
True
{'Bob', 'Henry', 'David', 'Eve', 'Alice'}
[1, 2, 3, 4]
```

```
# Task 13

import numpy as np

a1 = np.arange(0, 21, 2)
print(a1)
print(a1.shape, a1.dtype)

a2 = np.linspace(0, 1, 6)
print(a2)
print(a2.shape, a2.dtype)

a3 = np.zeros((3, 4))
print(a3)
print(a3.shape, a3.dtype)

a4 = np.ones((2, 5), dtype="int16")
print(a4)
print(a4.shape, a4.dtype)

a5 = np.eye(4)
print(a5)
print(a5.shape, a5.dtype)
```

```
a6 = np.random.randint(1, 101, size=(3, 3))
print(a6)
print(a6.shape, a6.dtype)
```

```
[ 0  2  4  6  8 10 12 14 16 18 20]
(11,) int64
[0.  0.2 0.4 0.6 0.8 1. ]
(6,) float64
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
(3, 4) float64
[[1 1 1 1 1]
 [1 1 1 1 1]]
(2, 5) int16
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
(4, 4) float64
[[65 29 68]
 [20 35 11]
 [99 84 21]]
(3, 3) int64
```

```
# Task 14

import numpy as np

A = np.arange(1, 26).reshape(5, 5)

print(A[2, 3])

print(A[2])

print(A[:, 1])

print(A[:3, :3])

print(A[-2:, -2:])

print(A.T)
```

```
print(A.reshape(25))
print(A.reshape(25, 1))
```

```
14
[11 12 13 14 15]
[ 2  7 12 17 22]
[[ 1  2  3]
 [ 6  7  8]
 [11 12 13]]
[[19 20]
 [24 25]]
[[ 1  6 11 16 21]
 [ 2  7 12 17 22]
 [ 3  8 13 18 23]
 [ 4  9 14 19 24]
 [ 5 10 15 20 25]]
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
 25]
[[ 1]
 [ 2]
 [ 3]
 [ 4]
 [ 5]
 [ 6]
 [ 7]
 [ 8]
 [ 9]
 [10]
 [11]
 [12]
 [13]
 [14]
 [15]
 [16]
 [17]
 [18]
 [19]
 [20]
 [21]
 [22]
 [23]
 [24]
 [25]]
```

```
# Task 15
```

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])
b = np.array([1, 2, 3, 4, 5])

print(a + b)
print(a - b)
print(a * b)
print(a / b)

print(np.sqrt(a))

mask = a > 25
print(mask)

print(a[mask])

M = np.array([[1, 2], [3, 4]])
N = np.array([[5, 6], [7, 8]])

print(M * N)

print(M @ N)

print(M.sum())
print(M.sum(axis=1))
print(M.sum(axis=0))
```

```
[11 22 33 44 55]
[ 9 18 27 36 45]
[ 10  40  90 160 250]
[10. 10. 10. 10. 10.]
[3.16227766 4.47213595 5.47722558 6.32455532 7.07106781]
[False False  True  True  True]
[30 40 50]
[[ 5 12]
 [21 32]]
[[19 22]
 [43 50]]
10
[3 7]
[4 6]
```

```
# Task 16
```

```
import pandas as pd
from sklearn.datasets import load_iris

data = {
    "Name": ["Ali", "Sara", "John", "Fatima", "Chris"],
    "Age": [23, 21, 25, 22, 24],
    "Score": [88, 92, 78, 95, 83],
    "Passed": [True, True, True, True, True],
    "City": ["Lahore", "Karachi", "London", "Islamabad", "New York"]
}

df = pd.DataFrame(data)

print(df.shape)
print(df.columns)
print(df.dtypes)

print(df.head(3))
print(df.tail(2))

print(df.describe())

iris = load_iris(as_frame=True)
iris_df = iris.frame
iris_df["species"] = iris.target_names[iris.target]

print(iris_df["species"].value_counts())

print(iris_df.groupby("species").mean(numeric_only=True))
```

```
(5, 5)
Index(['Name', 'Age', 'Score', 'Passed', 'City'], dtype='object')
Name      object
Age        int64
Score      int64
Passed      bool
City      object
dtype: object
   Name  Age  Score  Passed     City
0   Ali   23     88    True   Lahore
1  Sara   21     92    True  Karachi
2  John   25     78    True   London
     Name  Age  Score  Passed       City
3  Fatima   22     95    True  Islamabad
```

```
4   Chris   24     83    True   New York
             Age      Score
count   5.000000   5.00000
mean   23.000000  87.20000
std     1.581139   6.83374
min    21.000000  78.00000
25%    22.000000  83.00000
50%    23.000000  88.00000
75%    24.000000  92.00000
max    25.000000  95.00000
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
            sepal length (cm)  sepal width (cm)  petal length (cm)  \
species
setosa                  5.006             3.428              1.462
versicolor              5.936             2.770              4.260
virginica               6.588             2.974              5.552

            petal width (cm)  target
species
setosa                 0.246     0.0
versicolor             1.326     1.0
virginica              2.026     2.0
```

```
# Task 17

from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df['species'] = iris.target_names[iris.target]

print(iris_df[['sepal length (cm)', 'petal length (cm)']].head())

print(iris_df.iloc[50:55])

versicolor_df = iris_df[iris_df['species'] == 'versicolor']
print(len(versicolor_df))

filtered_df = iris_df[(iris_df['petal length (cm)'] > 5.0) & (iris_df['species'] == 'virginica')]
print(filtered_df)
```

```
iris_df['petal_area'] = iris_df['petal length (cm)'] * iris_df['petal width (cm)']
print(iris_df.head())

sorted_df = iris_df.sort_values('petal_area', ascending=False)
print(sorted_df.head())
```

```
   sepal length (cm)  petal length (cm)
0                5.1                1.4
1                4.9                1.4
2                4.7                1.3
3                4.6                1.5
4                5.0                1.4
    sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
50                7.0               3.2                4.7               1.4
51                6.4               3.2                4.5               1.5
52                6.9               3.1                4.9               1.5
53                5.5               2.3                4.0               1.3
54                6.5               2.8                4.6               1.5

      species
50  versicolor
51  versicolor
52  versicolor
53  versicolor
54  versicolor
50
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
100                6.3               3.3                6.0               2.5
101                5.8               2.7                5.1               1.9
102                7.1               3.0                5.9               2.1
103                6.3               2.9                5.6               1.8
104                6.5               3.0                5.8               2.2
105                7.6               3.0                6.6               2.1
107                7.3               2.9                6.3               1.8
108                6.7               2.5                5.8               1.8
109                7.2               3.6                6.1               2.5
110                6.5               3.2                5.1               2.0
111                6.4               2.7                5.3               1.9
112                6.8               3.0                5.5               2.1
114                5.8               2.8                5.1               2.4
115                6.4               3.2                5.3               2.3
116                6.5               3.0                5.5               1.8
117                7.7               3.8                6.7               2.2
118                7.7               2.6                6.9               2.3
120                6.9               3.2                5.7               2.3
122                7.7               2.8                6.7               2.0
```

```
124             6.7             3.3             5.7             2.1
125             7.2             3.2             6.0             1.8
128             6.4             2.8             5.6             2.1
129             7.2             3.0             5.8             1.6
130             7.4             2.8             6.1             1.9
131             7.9             3.8             6.4             2.0
132             6.4             2.8             5.6             2.2
133             6.3             2.8             5.1             1.5
134             6.1             2.6             5.6             1.4
135             7.7             3.0             6.1             2.3
136             6.3             3.4             5.6             2.4
137             6.4             3.1             5.5             1.8
139             6.9             3.1             5.4             2.1
140             6.7             3.1             5.6             2.4
141             6.9             3.1             5.1             2.3
142             5.8             2.7             5.1             1.9
143             6.8             3.2             5.9             2.3
144             6.7             3.3             5.7             2.5
```

```
# Task 18

import numpy as np
import pandas as pd

students = [
    {"name": "Alice", "scores": [85, 90, 78], "city": "Lahore"},
    {"name": "Bob", "scores": [60, 55, 70], "city": "Karachi"},
    {"name": "Carol", "scores": [92, 88, 95], "city": "Islamabad"},
    {"name": "David", "scores": [70, 65, 80], "city": "Lahore"},
    {"name": "Eve", "scores": [50, 45, 55], "city": "Karachi"},
    {"name": "Frank", "scores": [88, 92, 84], "city": "Islamabad"},
    {"name": "Grace", "scores": [75, 80, 78], "city": "Lahore"},
]

for s in students:
    s["average"] = round(sum(s["scores"]) / len(s["scores"]), 2)
    print(f"{s['name']}: {s['average']}")

grades = {
    s["name"]: "A" if s["average"] >= 85 else "B" if s["average"] >= 70 else "C" if s["average"] >= 55 else "F"
    for s in students
}
print(grades)

top_students = [s["name"] for s in students if s["average"] >= 80]
```

```
print(top_students)

unique_cities = {s["city"] for s in students}
print(unique_cities)

scores_matrix = np.array([s["scores"] for s in students])
print(scores_matrix.mean(), scores_matrix.min(), scores_matrix.max())
print(scores_matrix.mean(axis=1))
print(scores_matrix.mean(axis=0))

df = pd.DataFrame([
    {
        "Name": s["name"],
        "Average": s["average"],
        "Grade": grades[s["name"]],
        "City": s["city"]
    }
    for s in students
])
print(df)
print(df.groupby("City")["Average"].mean())
```

```
Alice: 84.33
Bob: 61.67
Carol: 91.67
David: 71.67
Eve: 50.0
Frank: 88.0
Grace: 77.67
{'Alice': 'B', 'Bob': 'C', 'Carol': 'A', 'David': 'B', 'Eve': 'F', 'Frank': 'A', 'Grace': 'B'}
['Alice', 'Carol', 'Frank']
{'Islamabad', 'Lahore', 'Karachi'}
75.0 45 95
[84.33333333 61.66666667 91.66666667 71.66666667 50.         88.
 77.66666667]
[74.28571429 73.57142857 77.14285714]
    Name  Average Grade       City
0  Alice    84.33     B     Lahore
1    Bob    61.67     C    Karachi
2  Carol    91.67     A  Islamabad
3  David    71.67     B     Lahore
4    Eve    50.00     F    Karachi
5  Frank    88.00     A  Islamabad
6  Grace    77.67     B     Lahore
City
```

```
Islamabad    89.835
Karachi      55.835
Lahore       77.890
Name: Average, dtype: float64
```